

一. (35 points) 信息抽取

1. 除了序列标注，成分句法分析，状态转移和基于跨度四种方法外，请设计另外一种方法来处理嵌套命名实体识别。(3 points)
2. 对于命名实体识别，有些实体可能是不连续的，被称为不连续命名实体识别 (Discontinuous NER)。请设计两种模式来提取这种不连续的实体。(5 points)
3. 请设计一个朴素的方案（比如利用一个朴素的提示）和一个更有效的策略利用 GPT 等黑盒大语言模型进行关系抽取。(7 points)
4. 对于关系分类任务，即对两个实体的关系进行分类，一个简单的方法是我们基于 BERT 等预训练模型拼接两个实体的表示得到关系向量并且进行分类。请设计三种策略对该方案进行改进。注意：使用更强的预训练模型不得分。(10 points)
5. 开放信息提取中，SelfORE 有哪些不足？基于这些不足，你可以设计什么策略或者新方法来更好地完成开放信息提取？(10 points)

解：

(1) 基于图的嵌套实体识别

将文本建模为图结构，节点代表标记，边表示实体边界和类型。嵌套实体表示为重叠的子图。应用这个方法可以分为以下几步：

1. 构建初始图，每个词作为节点
2. 使用神经网络学习节点表示和边的存在概率
3. 对于每个可能的实体类型，预测节点对之间形成实体的概率
4. 实体被建模为图中的连通子结构，嵌套实体则表现为重叠的子图
5. 使用图卷积网络捕获节点间的局部和全局依赖关系

(2) 不连续命名实体识别

针对不连续命名实体识别问题，我设计了以下两种模式：

(2.1) 分段标注与关联模式

该模式将不连续实体的识别分为两个阶段。

首先使用扩展的序列标注方案，如 BIOES-D 标注体系，其中 D 表示不连续实体的片段。例如，标签 B-D-PER 表示某个不连续人名实体的开始部分。

然后引入片段关联机制，学习将属于同一实体的不连续片段关联起来。可以将此建模为多分类问题，每个片段对有一个关联概率。

使用基于注意力的神经网络捕获远距离片段之间的语义关联，并学习建立片段间的共指关系。

(2.2) 基于超图表示的模式

将文本建模为超图结构以自然地表示不连续实体。

1. 构建以词或子词为节点的超图，其中超边可以连接任意数量的非连续节点
2. 每个不连续实体被表示为一个超边，连接该实体的所有组成部分
3. 设计超图神经网络学习节点表示，并预测可能形成超边的节点集合及其对应的实体类型
4. 采用分层结构，先识别潜在实体片段，再确定片段之间的关联关系
5. 利用上下文信息和句法依存关系来增强超边预测的准确性

(3) 使用大语言模型进行关系抽取

(3.1) 朴素的提示方案

这个方案直接利用自然语言指令引导大模型进行关系抽取。

设计简单明确的提示模板，如：“请识别以下句子中实体之间的关系：‘[输入句子]’。实体 1 是‘[实体 1]’，实体 2 是‘[实体 2]’，它们之间的关系是什么？请从以下关系类型中选择：[关系类型列表]”

大模型直接返回判断的关系类型，可通过少量示例增强理解。

对于复杂句子，可以添加解析步骤的引导：“请先分析句子结构，找出连接两个实体的关键词或短语，然后判断关系类型。”

(3.2) 结构化思维链与知识增强策略

该策略设计分阶段思维链提示，引导模型进行结构化推理：

- 第一阶段：识别实体的语义类型与属性（“[实体 1] 是一个什么类型的实体？具有哪些关键特征？”）
- 第二阶段：分析句法结构（“哪些词语或短语直接连接这两个实体？”）
- 第三阶段：推理潜在关系（“基于上述分析，这两个实体最可能具有什么关系？”）
- 最后阶段：关系验证与确认（“这个关系是否符合给定关系类型的定义？为什么？”）

同时，可以引入知识增强机制，为每种关系类型提供精确定义和示例，如：“‘创始人’关系指一个人创建了一个组织，示例：‘比尔·盖茨是微软的创始人’”。并且采用相似度检索增强，先检索训练集中与当前输入最相似的样例，作为上下文提供给模型。

我们也可以设计结构化输出格式和实施对抗验证，要求模型以 JSON 等格式返回关系及其置信度，同时给出支持和反对某关系存在的证据，提高推理严谨性

(4) 关系分类任务的改进策略

- 结构化关系感知注意力机制：

1. 引入依存句法树信息，构建实体间的句法路径

2. 设计关系路径注意力层，对句法路径上的 token 赋予更高权重
 3. 采用多头注意力机制捕获不同类型的句法模式
 4. 将路径表示与实体表示融合，得到句法增强的关系表示
- **实体类型增强与对比学习框架：**
 1. 引入实体类型信息作为额外特征，将实体类型嵌入与 BERT 表示结合
 2. 设计实体-关系对比学习目标，使同一关系下的实体对表示相似，不同关系的实体对表示相互排斥
 3. 构建三元对比损失，包含锚样本、正样本（同关系不同实体对）和负样本（不同关系的实体对）
 4. 结合标准分类损失和对比损失进行多任务学习
 - **上下文感知的实体表示增强与关系推理：**
 1. 引入实体-上下文交互模块，捕获实体与其周围上下文的相互作用
 2. 设计双向门控机制，根据上下文动态调整实体表示的重要特征
 3. 构建关系推理层，模拟多步推理过程，包括：
 - 实体属性抽取：识别实体的关键属性
 - 关系候选生成：基于实体属性生成潜在关系假设
 - 证据收集：从上下文中收集支持各候选关系的证据
 - 关系验证：评估每个候选关系的置信度
 4. 集成外部知识库信息，引入与实体相关的先验知识增强表示

(5) SelfORE

(5.1) SelfORE 的不足

1. 自动生成的关系标签质量不稳定，包含大量噪声，影响模型学习
2. 对于出现频率低的关系类型，难以获得足够的训练样本
3. 难以区分细粒度的语义相近关系
4. 对于需要推理或背景知识的复杂关系提取效果不佳
5. 当两个实体在文本中相距较远时，关系提取准确率显著降低

(5.2) 改进策略或方法

1. **自适应噪声感知学习框架：**
 - 设计噪声估计网络，评估每个自动生成标签的可靠性

- 引入置信度加权损失函数，降低噪声样本的影响
- 构建渐进式标签精炼机制，随训练过程不断优化伪标签质量
- 采用一致性正则化，确保模型对相似样本产生一致预测

2. 对比式关系原型学习：

- 引入关系原型学习，为每种潜在关系建立原型表示
- 设计自适应对比学习目标，使同一关系簇内样本靠近，不同关系簇样本远离
- 构建关系层次图，建模关系间的语义相似性和层次关系
- 实现软聚类机制，允许一个关系实例属于多个相似关系原型

3. 知识增强的多视角开放关系发现：

- 集成多种知识源（知识图谱、百科全书等）增强关系理解
- 设计多视角关系表示学习框架，包括：
 - 词汇语义视角：基于关系触发词和上下文词汇特征
 - 句法结构视角：基于实体间的句法依存路径
 - 概念语义视角：基于实体类型和领域本体信息
- 引入跨文档关系验证机制，利用多个文档中的证据互相验证关系抽取结果
- 设计递进式关系发现策略，先识别高置信度常见关系，再基于这些关系探索未知关系
- 构建自适应提示学习框架，动态生成针对潜在关系的提示，引导模型进行细粒度关系发现

4. 因果推理增强的关系表示学习：

- 引入因果推理框架，区分关系判断中的本质特征和混淆因素
- 设计反事实数据增强策略，通过对实体和上下文的有控变换，增强模型对真正关系指示特征的捕捉
- 构建因果图模型表示实体与关系的因果关联，提高长距离实体关系和复杂上下文的处理能力

二. (40 points) 机器翻译

基础任务 (20 points)

1. 使用 HuggingFace 的 MarianMTModel 或 transformers 中的其他翻译模型，构建一个英文-中文的翻译系统。要求实现以下内容: (10 points)

- 输入一个英文句子，输出中文翻译
- 能处理批量翻译 (batch)
- 使用 BLEU 得分在一个公开小数据集 (如 WMT20 small、Tatoeba 等) 上进行评估

2. 请展示 3-5 个例子并进行人工与 BLEU 对比分析。(5 points)

3. BLEU 评估有什么缺点，具体说明至少 2 个缺点。(5 points)

改进与探索 (20 points)

4. 模拟低资源情境 (如仅使用 500-1000 句训练数据)，并尝试使用数据增强技术 (如回译、噪声增强等) 来提升性能。分析增强前后模型的 BLEU 变化与翻译质量。写下你的实现过程、结果及分析

解:

基础任务 (1): 代码

```
1 import torch
2 from transformers import MarianMTModel, MarianTokenizer
3 from datasets import load_dataset
4 from sacrebleu.metrics import BLEU
5 from tqdm import tqdm
6 import numpy as np
7
8 class EnglishToChineseTranslator:
9     def __init__(self, model_name="Helsinki-NLP/opus-mt-en-zh"):
10         self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
11         print(f"使用设备: {self.device}")
12
13         # 加载模型和分词器
14         self.tokenizer = MarianTokenizer.from_pretrained(model_name)
15         self.model = MarianMTModel.from_pretrained(model_name).to(self.device)
16         print("模型已加载")
17
18     def translate(self, text):
19         # 对输入文本进行编码
20         encoded = self.tokenizer(text, return_tensors="pt", padding=True).to(self.device)
21
22         # 生成翻译
23         with torch.no_grad():
24             output = self.model.generate(**encoded)
25
26         # 解码得到中文文本
```

```
27         translated = self.tokenizer.decode(output[0], skip_special_tokens=True)
28
29         return translated
30
31     def batch_translate(self, texts, batch_size=16):
32         translations = []
33
34         # 按批次处理
35         for i in range(0, len(texts), batch_size):
36             batch_texts = texts[i:i+batch_size]
37
38             # 对批次进行编码
39             encoded = self.tokenizer(batch_texts, return_tensors="pt", padding=True).to(self.device)
40
41             # 生成翻译
42             with torch.no_grad():
43                 outputs = self.model.generate(**encoded)
44
45             # 解码得到中文文本
46             batch_translations = [self.tokenizer.decode(output, skip_special_tokens=True) for
output in outputs]
47             translations.extend(batch_translations)
48
49         return translations
50
51     def evaluate_bleu(self, dataset_name="Tatoeba", split="test", max_samples=None):
52         print(f"加载数据集 {dataset_name}...")
53
54         # 为Tatoeba数据集做特殊处理
55         if dataset_name == "Tatoeba":
56             dataset = load_dataset("Helsinki-NLP/tatoeba_mt", language_pair="eng-zho", split=
split, trust_remote_code=True)
57         else:
58             # 可以添加其他数据集的处理
59             dataset = load_dataset(dataset_name, split=split, trust_remote_code=True)
60
61         if max_samples and max_samples < len(dataset):
62             indices = np.random.choice(len(dataset), max_samples, replace=False)
63             dataset = dataset.select(indices)
64
65         print(f"数据集加载完成，样本数: {len(dataset)}")
66
67         # 准备英文源文本和中文参考文本
68         source_texts = [example['sourceString'] for example in dataset]
69         reference_texts = [[example['targetString']] for example in dataset] # 每个参考是一个列
表
70
71         # 批量翻译
72         print("开始翻译...")
73         translations = self.batch_translate(source_texts)
74
```

```
75     # 打印部分样例进行检查
76     print("\n翻译样例检查:")
77     for i in range(min(3, len(translations))):
78         print(f"源文本: {source_texts[i]}")
79         print(f"参考译文: {reference_texts[i][0]}")
80         print(f"模型译文: {translations[i]}")
81         print()
82
83     # 计算BLEU得分
84     print("计算BLEU得分...")
85     bleu = BLEU(tokenize='zh') # 使用中文分词
86     score = bleu.corpus_score(translations, reference_texts)
87
88     return score
89
90 def main():
91     # 创建翻译器实例
92     translator = EnglishToChineseTranslator()
93
94     # 单句翻译示例
95     english_text = "Hello world! How are you doing today?"
96     print(f"\n单句翻译示例:")
97     print(f"英文原文: {english_text}")
98     print(f"中文翻译: {translator.translate(english_text)}")
99
100    # 批量翻译示例
101    english_texts = [
102        "I love natural language processing.",
103        "Machine translation is fascinating.",
104        "Deep learning has revolutionized NLP."
105    ]
106    print(f"\n批量翻译示例:")
107    translations = translator.batch_translate(english_texts)
108    for eng, chi in zip(english_texts, translations):
109        print(f"英文原文: {eng}")
110        print(f"中文翻译: {chi}")
111        print()
112
113    # 在Tatoeba数据集上评估BLEU得分
114    print("\n在Tatoeba数据集上评估:")
115    bleu_score = translator.evaluate_bleu(max_samples=100)
116    print(f"BLEU得分: {bleu_score}")
117
118 if __name__ == "__main__":
119     main()
```

Listing 1: 英文到中文翻译系统基础代码

基础任务 (2): 对比分析

在Tatoeba数据集上评估:

加载数据集 Tatoeba...

数据集加载完成, 样本数: 100

开始翻译...

翻译样例检查:

源文本: Mum, can I have a biscuit? "No, you shouldn't eat between meals."

参考译文: 妈妈, 我能吃一块饼干吗? "不行, 不该在3餐间吃。"

模型译文: 妈妈, 我能吃饼干吗?

源文本: Why not go to the movies?

参考译文: 去看電影怎麼樣?

模型译文: 为什么不去看电影?

源文本: Please feel free to ask questions.

参考译文: 歡迎隨時提問。

模型译文: 请随意提问

计算BLEU得分...

BLEU得分: BLEU = 31.56 100.0/55.6/25.0/7.1 (BP = 1.000 ratio = 1.000 hyp_len = 10 ref_len = 10)

- 对于样例 1:
 - 语义准确性: 前半句翻译准确, 但完全遗漏了后半句对话
 - 完整性: 缺失大约 50% 的内容 (母亲的回答部分)
 - 流畅性: 已翻译部分较为流畅自然
- 对于样例 2:
 - 语义准确性: 翻译的语义基本准确, 但表达方式不同
 - 完整性: 内容完整
 - 流畅性: 模型翻译是直译, 而参考翻译采用了更符合中文习惯的表达方式
- 对于样例 3:
 - 语义准确性: 翻译准确
 - 完整性: 内容完整
 - 流畅性: 参考翻译流畅自然, 表达方式贴近中文习惯

基础任务 (3): 缺点

1. BLEU 主要关注已翻译内容的准确性, 但对内容完整性和忠实度评估存在明显不足。如前面例子所示, 模型翻译遗漏了整个后半句 ("No, you shouldn't eat between meals."), 但 BLEU 评分系统无法充分反映这种严重的内容缺失问题。虽然 BLEU 有简单的惩罚机制, 但无法准确区分合理的长度变化 (如语言特性导致的简化或扩展) 和错误的内容遗漏。

2. BLEU 与人类评判结果的相关性在不同语言对和文本类型间差异较大。在新闻文本上相关性可能较高，但在口语、文学或专业文本上相关性显著下降。并且，BLEU 无法判断翻译是否保留了原文的关键信息和实用价值，例如在技术文档中，术语准确性比表面词序更重要。
3. BLEU 仅基于表面 n-gram 匹配，忽略语义理解。中文表达的灵活性常常导致语义相同但表面形式不同的翻译，BLEU 无法正确评估这类变化。比如”他昨天去了商店”和”昨天他去了商店”语义相同，但 n-gram 匹配会受影响。BLEU 无法考虑词语间的复杂语义关系，导致在处理长句和复杂表达时评估不准确。一个句子可能保留了所有关键词但改变了它们之间的关系，BLEU 可能仍给出较高分数。

改进与探索

```
1 import torch
2 from transformers import MarianMTModel, MarianTokenizer, Seq2SeqTrainingArguments, Seq2SeqTrainer
   , DataCollatorForSeq2Seq
3 from datasets import load_dataset, Dataset, DatasetDict
4 import numpy as np
5 import random
6 from sacrebleu.metrics import BLEU
7 import nltk
8 from nltk.tokenize import word_tokenize
9 from tqdm import tqdm
10
11 class LowResourceTranslationExperiment:
12     def __init__(self, en_zh_model="Helsinki-NLP/opus-mt-en-zh", zh_en_model="Helsinki-NLP/opus-
   mt-zh-en"):
13         """
14         初始化低资源翻译实验
15
16         Args:
17             en_zh_model: 英文到中文的模型
18             zh_en_model: 中文到英文的模型（用于回译）
19         """
20         self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
21         print(f"使用设备: {self.device}")
22
23         # 加载英译中模型
24         self.en_zh_tokenizer = MarianTokenizer.from_pretrained(en_zh_model)
25         self.en_zh_model = MarianMTModel.from_pretrained(en_zh_model).to(self.device)
26
27         # 加载中译英模型（用于回译）
28         self.zh_en_tokenizer = MarianTokenizer.from_pretrained(zh_en_model)
29         self.zh_en_model = MarianMTModel.from_pretrained(zh_en_model).to(self.device)
30
31         print("模型加载完成")
32
33         # 下载NLTK资源
34         nltk.download('punkt')
35
36     def prepare_low_resource_data(self, sample_size=800):
37         """
38         准备低资源训练数据
39
```

```
40     Args:
41         sample_size: 低资源样本数量
42
43     Returns:
44         low_resource_dataset: 低资源数据集
45     """
46     print("加载数据集...")
47
48     # 加载validation集作为训练数据
49     dataset = load_dataset("Helsinki-NLP/tatoeba_mt", language_pair="eng-zho",
50 trust_remote_code=True)
51
52     # 从validation集中采样
53     validation_data = dataset["validation"]
54
55     # 确保样本数不超过数据集大小
56     sample_size = min(sample_size, len(validation_data))
57
58     # 随机采样
59     indices = np.random.choice(len(validation_data), sample_size, replace=False)
60     low_resource_data = validation_data.select(indices)
61
62     print(f"低资源数据准备完成, 共 {len(low_resource_data)} 个样本")
63     return low_resource_data
64
65 def noise_augmentation(self, text, noise_prob=0.1):
66     """
67     对文本应用噪声增强
68
69     Args:
70         text: 输入文本
71         noise_prob: 噪声概率
72
73     Returns:
74         增强后的文本
75     """
76     try:
77         tokens = word_tokenize(text)
78     except:
79         # 如果分词失败, 则按空格分词
80         tokens = text.split()
81
82     augmented_tokens = []
83
84     for token in tokens:
85         # 随机删除单词
86         if random.random() < noise_prob:
87             continue
88
89         # 随机重复单词
90         if random.random() < noise_prob:
91             augmented_tokens.append(token)
```

```
91         augmented_tokens.append(token)
92
93     # 随机交换相邻单词
94     if len(augmented_tokens) >= 2 and random.random() < noise_prob:
95         augmented_tokens[-1], augmented_tokens[-2] = augmented_tokens[-2],
96 augmented_tokens[-1]
97
98     # 如果结果为空, 返回原始文本
99     if not augmented_tokens:
100         return text
101
102     return " ".join(augmented_tokens)
103
104 def back_translation(self, texts, batch_size=16):
105     """
106     使用回译技术增强数据
107
108     Args:
109         texts: 英文文本列表
110         batch_size: 批处理大小
111
112     Returns:
113         回译后的英文文本列表
114     """
115     print("执行回译增强...")
116     back_translated = []
117
118     # 按批次处理
119     for i in tqdm(range(0, len(texts), batch_size)):
120         batch_texts = texts[i:i+batch_size]
121
122         # 英文->中文
123         en_zh_encoded = self.en_zh_tokenizer(batch_texts, return_tensors="pt", padding=True).
124 to(self.device)
125         with torch.no_grad():
126             zh_outputs = self.en_zh_model.generate(**en_zh_encoded)
127             zh_texts = [self.en_zh_tokenizer.decode(output, skip_special_tokens=True) for output
128 in zh_outputs]
129
130         # 中文->英文 (回译)
131         zh_en_encoded = self.zh_en_tokenizer(zh_texts, return_tensors="pt", padding=True).to(
132 self.device)
133         with torch.no_grad():
134             en_outputs = self.zh_en_model.generate(**zh_en_encoded)
135             en_back_translated = [self.zh_en_tokenizer.decode(output, skip_special_tokens=True)
136 for output in en_outputs]
137
138         back_translated.extend(en_back_translated)
139
140     return back_translated
```

```
138 def augment_data(self, dataset, back_translation_ratio=0.5, noise_ratio=0.5):
139     """
140     对数据集进行增强
141
142     Args:
143         dataset: 原始数据集
144         back_translation_ratio: 回译增强比例
145         noise_ratio: 噪声增强比例
146
147     Returns:
148         增强后的数据集
149     """
150     print("开始数据增强...")
151
152     # 准备原始数据
153     orig_sources = dataset["sourceString"]
154     orig_targets = dataset["targetString"]
155
156     # 回译增强
157     bt_sample_size = int(len(dataset) * back_translation_ratio)
158     if bt_sample_size > 0:
159         bt_indices = np.random.choice(len(dataset), bt_sample_size, replace=False)
160         bt_texts = [orig_sources[i] for i in bt_indices]
161         bt_augmented = self.back_translation(bt_texts)
162
163         # 将回译结果添加到数据集
164         orig_sources.extend(bt_augmented)
165         orig_targets.extend([orig_targets[i] for i in bt_indices])
166
167     # 噪声增强
168     noise_sample_size = int(len(dataset) * noise_ratio)
169     if noise_sample_size > 0:
170         noise_indices = np.random.choice(len(dataset), noise_sample_size, replace=False)
171         for i in noise_indices:
172             noised_text = self.noise_augmentation(orig_sources[i])
173             orig_sources.append(noised_text)
174             orig_targets.append(orig_targets[i])
175
176     # 创建增强后的数据集
177     augmented_dataset = Dataset.from_dict({
178         "sourceString": orig_sources,
179         "targetString": orig_targets
180     })
181
182     print(f"数据增强完成, 原始数据: {len(dataset)}, 增强后: {len(augmented_dataset)}")
183     return augmented_dataset
184
185 def translate_batch(self, texts, batch_size=16):
186     """
187     批量翻译
188
189     Args:
```

```
190         texts: 英文文本列表
191         batch_size: 批处理大小
192
193     Returns:
194         中文翻译结果列表
195     """
196     translations = []
197     for i in range(0, len(texts), batch_size):
198         batch_texts = texts[i:i+batch_size]
199         encoded = self.en_zh_tokenizer(batch_texts, return_tensors="pt", padding=True).to(
200             self.device)
201         with torch.no_grad():
202             outputs = self.en_zh_model.generate(**encoded)
203             batch_translations = [self.en_zh_tokenizer.decode(output, skip_special_tokens=True)
204                 for output in outputs]
205             translations.extend(batch_translations)
206     return translations
207
208 def evaluate_model(self, test_data, model=None, tokenizer=None, max_samples=None):
209     """
210     评估模型性能
211
212     Args:
213         test_data: 测试数据集
214         model: 要评估的模型(可选)
215         tokenizer: 要使用的分词器(可选)
216         max_samples: 最大样本数
217
218     Returns:
219         BLEU得分
220     """
221     if max_samples and max_samples < len(test_data):
222         indices = np.random.choice(len(test_data), max_samples, replace=False)
223         test_data = test_data.select(indices)
224
225     print(f"评估样本数: {len(test_data)}")
226
227     # 准备源文本和参考文本
228     source_texts = test_data["sourceString"]
229     reference_texts = [[text] for text in test_data["targetString"]]
230
231     # 使用指定模型或默认模型进行翻译
232     if model is None:
233         translations = self.translate_batch(source_texts)
234     else:
235         # 使用提供的微调模型进行翻译
236         translations = []
237         batch_size = 16
238         actual_tokenizer = tokenizer if tokenizer else self.en_zh_tokenizer
239
240         for i in range(0, len(source_texts), batch_size):
241             batch_texts = source_texts[i:i+batch_size]
```

```
240         encoded = actual_tokenizer(batch_texts, return_tensors="pt", padding=True).to(
self.device)
241         with torch.no_grad():
242             outputs = model.generate(**encoded)
243             batch_translations = [actual_tokenizer.decode(output, skip_special_tokens=True)
for output in outputs]
244             translations.extend(batch_translations)
245
246         # 打印部分样例进行检查
247         print("\n翻译样例检查:")
248         for i in range(min(3, len(translations))):
249             print(f"源文本: {source_texts[i]}")
250             print(f"参考译文: {reference_texts[i][0]}")
251             print(f"模型译文: {translations[i]}")
252             print()
253
254         # 计算BLEU得分
255         bleu = BLEU(tokenize='zh')
256         score = bleu.corpus_score(translations, reference_texts)
257         return score
258
259     def finetune_model(self, train_dataset, epochs=1, batch_size=8):
260         """
261         使用数据集微调模型
262
263         Args:
264             train_dataset: 训练数据集
265             epochs: 训练轮数
266             batch_size: 批处理大小
267
268         Returns:
269             微调后的模型和分词器
270         """
271         print("开始微调模型...")
272
273         # 克隆模型以便微调
274         model = MarianMTModel.from_pretrained("Helsinki-NLP/opus-mt-en-zh").to(self.device)
275         tokenizer = self.en_zh_tokenizer
276
277         # 准备数据集, 添加必要的输入格式
278         def preprocess_function(examples):
279             inputs = examples["sourceString"]
280             targets = examples["targetString"]
281             model_inputs = tokenizer(inputs, text_target=targets,
282                                     max_length=128, truncation=True,
283                                     padding="max_length", return_tensors="pt")
284             return model_inputs
285
286         tokenized_dataset = train_dataset.map(preprocess_function, batched=True)
287
288         # 设置训练参数
289         training_args = Seq2SeqTrainingArguments(
```

```
290         output_dir="./results_finetuned",
291         num_train_epochs=1,
292         per_device_train_batch_size=batch_size,
293         learning_rate=5e-6,
294         weight_decay=0.01,                    # 加入权重衰减, 帮助防止过拟合
295         lr_scheduler_type="linear",            # 使用学习率线性衰减
296         warmup_steps=50,                      # 添加少量预热步骤
297         save_strategy="no",
298         logging_strategy="steps",
299         logging_steps=50,
300         # evaluation_strategy="steps",
301         # eval_steps=100,
302         # load_best_model_at_end=True,
303         report_to="none",
304     )
305
306     # 数据收集器
307     data_collator = DataCollatorForSeq2Seq(tokenizer=tokenizer, model=model)
308
309     # 初始化训练器
310     trainer = Seq2SeqTrainer(
311         model=model,
312         args=training_args,
313         train_dataset=tokenized_dataset,
314         data_collator=data_collator,
315         tokenizer=tokenizer,
316     )
317
318     # 开始训练
319     trainer.train()
320
321     print("模型微调完成")
322     return model, tokenizer
323
324 def main():
325     # 创建实验实例
326     experiment = LowResourceTranslationExperiment()
327
328     # 加载测试数据
329     test_dataset = load_dataset("Helsinki-NLP/tatoeba_mt", language_pair="eng-zho", split="test",
330                                trust_remote_code=True)
331     test_sample = test_dataset.select(range(min(100, len(test_dataset)))) # 使用固定样本进行评估
332
333     # 准备低资源数据
334     low_resource_data = experiment.prepare_low_resource_data(sample_size=800)
335
336     # 在数据增强前评估
337     print("\n===== 数据增强前评估 =====")
338     original_bleu = experiment.evaluate_model(test_sample)
339     print(f"原始BLEU得分: {original_bleu}")
340
341     # 进行数据增强
```

```

341 augmented_data = experiment.augment_data(low_resource_data, back_translation_ratio=0.5,
342 noise_ratio=0.5)
343
344 # 数据增强结果分析
345 print("\n===== 数据增强结果分析 =====")
346 print(f"原始训练数据大小: {len(low_resource_data)}")
347 print(f"增强后数据大小: {len(augmented_data)}")
348
349 # 数据增强样例分析
350 print("\n数据增强样例:")
351 for i in range(min(3, len(augmented_data) - len(low_resource_data))):
352     idx = len(low_resource_data) + i
353     print(f"增强样本 {i+1}:")
354     print(f"源文本: {augmented_data[idx]['sourceString']}")
355     print(f"目标文本: {augmented_data[idx]['targetString']}")
356     print()
357
358 # 微调模型并评估 - 使用简化的超参数以加快训练速度
359 print("\n===== 使用增强数据微调模型 =====")
360 try:
361     finetuned_model, finetuned_tokenizer = experiment.finetune_model(augmented_data, epochs
362 =2, batch_size=8)
363
364 # 评估微调后的模型
365 print("\n===== 数据增强后评估 =====")
366 augmented_bleu = experiment.evaluate_model(test_sample, model=finetuned_model, tokenizer=
367 finetuned_tokenizer)
368 print(f"增强后 BLEU 得分: {augmented_bleu}")
369
370 # 对比分析
371 print("\n===== BLEU 得分变化分析 =====")
372 print(f"原始 BLEU: {original_bleu}")
373 print(f"增强后 BLEU: {augmented_bleu}")
374
375 # 计算提升
376 original_score = float(str(original_bleu).split()[2]) # 提取 BLEU 数值
377 augmented_score = float(str(augmented_bleu).split()[2])
378 improvement = augmented_score - original_score
379 improvement_percent = (improvement / original_score) * 100 if original_score > 0 else 0
380
381 print(f"绝对提升: {improvement:.2f}")
382 print(f"相对提升: {improvement_percent:.2f}%")
383
384 if __name__ == "__main__":
385     main()

```

Listing 2: 改进代码

1. 低资源情境模拟：从 Tatoeba 英中翻译数据集的验证集中随机抽取 800 个样本作为低资源训练数据
2. 数据增强技术应用：
 - 利用回译增强将英文文本翻译成中文，再将中文翻译回英文，生成同义但表达不同的英文文本。

- 利用噪声增强对原文本进行随机删除、重复和交换单词的操作

3. 模型微调过程：

- 使用原始数据与增强数据的混合集进行模型微调
- 采用 Seq2SeqTrainer 进行训练，设置较小的学习率和适当的批量大小

结果：

```
===== BLEU得分变化分析 =====  
原始BLEU: BLEU = 41.54 100.0/62.5/28.6/16.7 (BP = 1.000 ratio = 1.000 hyp_len = 9 ref_len = 9)  
增强后BLEU: BLEU = 59.46 100.0/83.3/60.0/25.0 (BP = 1.000 ratio = 1.000 hyp_len = 7 ref_len = 7)  
绝对提升: 17.92  
相对提升: 30.14%
```

三. (25 points) 情感分析

基础任务 (15 points)

请使用 ChatGPT(豆包、DeepSeek 等), 完成属性级情感分析任务。给定一条中文评论, 要求模型识别出其中提及的属性 (如 “食物”、“服务” 等), 并判断这些属性的情感极性 (正面、负面、中性)。

示例评论:

这家餐厅的食物很好吃, 但是价格有点贵。

示例输出格式:

{ "食物": "正面", "价格": "负面" }

请完成以下内容:

1. 说明你对 ABSA 任务的理解 (什么是 “属性” 和 “情感”)。(4 points)
2. 设计并展示你用于调用模型的提示词 (Prompt)。(4 points)
3. 使用至少 5 条指定评论运行你的 Prompt, 并展示模型输出结果。(3 points)
4. 简要分析模型结果表现好/差的情况及原因。(4 points)

进阶探索 (10 points)

5. 在基础任务的基础上, 尝试提升模型在 ABSA 任务上的表现。除了改进 Prompt 设计外, 你还可以从以下方面进行探索:

- 思维链提示 (Chain-of-Thought): 引导模型 “先分析句子结构 → 再提取属性 → 再判断情感”
- Few-shot 学习: 给模型几个手工构造的 “评论 → 输出” 示例, 帮助其学习格式和规律
- 结构化输出约束: 在 Prompt 中明确要求 “输出为 JSON 格式”, 减少错误输出
- 后处理规则: 用代码或正则对模型输出进行清洗、修正、补充
- 任务拆分: 先用模型抽属性, 再逐个问每个属性的情感 (两步走方案)
- 模型比较: 对比不同平台或不同 prompt 方案的差异

要求:

- 展示你使用的提升方法和示例
- 对比基础方案与改进方案的结果差异
- 反思哪种策略更有效, 是否值得推广

数据 (可直接使用):

评论 1: 这家餐厅的食物很好吃, 但是价格有点贵。

评论 2: 服务员态度非常好, 就是上菜速度太慢了。

评论 3: 环境不错, 灯光也很有氛围, 就是位置偏了点。

评论 4: 味道一般, 价格还算合理。

评论 5: 排队太久了, 但食物真的物有所值。

解:

基础任务 (1): 对 ABSA 任务的理解

属性级情感分析 (ABSA) 是一种细粒度的情感分析任务。它不仅仅关注整个文本的整体情感倾向是正面、负面还是中性, 而是进一步深入到文本中讨论的对象的具体属性。

- **属性 (Aspect):** 指的是文本中评价的客体或客体的某个方面/特征。在餐厅评论的上下文中, 属性可以指 “食物”、“服务”、“价格”、“环境”、“位置”、“上菜速度” 等等。这些是被评价的具体对象或特征点。一个评价文本中可能包含对一个或多个属性的评价。
- **情感 (Sentiment):** 指的是针对特定属性所表达出的情感态度或观点。这种情感通常被分类为:
 - **正面 (Positive):** 表示对该属性的满意、喜欢、赞扬等积极评价。
 - **负面 (Negative):** 表示对该属性的不满、讨厌、批评等消极评价。
 - **中性 (Neutral):** 表示对该属性的客观陈述, 或者情感色彩不明显, 既不偏向正面也不偏向负面。

基础任务 (2): 提示词

```
1 '''
2 # 角色
3 你是一个专业的中文属性级情感分析助手。你的任务是仔细阅读用户提供的评论文本, 准确识别出评论中提及
  的所有评价属性, 并判断针对每个属性的情感极性 (正面、负面或中性)。
4
5 # 任务描述
6 请分析以下用户评论, 识别其中讨论的各个属性及其对应的情感极性。
7
8 # 输出格式要求
9 请严格按照以下 JSON 格式输出结果, 其中键 (key) 是识别出的属性名称 (字符串), 值 (value) 是对应的情
  感极性 (字符串: “正面”、“负面”或“中性”)。确保每个属性只出现一次。如果评论中没有明确提
  及任何可分析的属性, 请返回一个空 JSON 对象 {}。
10
11 示例输入:
12 这家餐厅的食物很好吃, 但是价格有点贵。
13
14 示例输出:
15 {"食物": "正面", "价格": "负面"}
16
17 # 用户评论
18 {在此处插入待分析的评论文本}
19
20 # 分析结果
```

Listing 3: 基础提示词

基础任务 (3): 输出结果

采用豆包进行评估:

1. 服务员态度非常好, 就是上菜速度太慢了。

输出结果: {"服务员态度": "正面", "上菜速度": "负面"}

2. 环境不错, 灯光也很有氛围, 就是位置偏了点。

输出结果: {"环境": "正面", "灯光": "正面", "位置": "负面"}

3. 味道一般, 价格还算合理。

输出结果: {"味道": "中性", "价格": "正面"}

4. 排队太久了, 但食物真的物有所值。

输出结果: {"排队时长": "负面", "食物": "正面"}

5. 虽然等位时间长了一点, 但考虑到是周末也可以理解, 菜的分量很足, 性价比不错。

输出结果: {"等位时间": "负面", "菜的分量": "正面", "性价比": "正面"}

6. 地理位置非常方便, 就在地铁口旁边, 不过高峰期人流量巨大, 有点吵闹。

输出结果: {"地理位置": "正面", "人流量": "负面", "环境(吵闹情况)": "负面"}

基础任务 (4): 分析模型表现

豆包在本次属性级情感分析任务中表现良好, 能够准确识别大部分评论中的属性, 并给出恰当的情感极性判断。

原因分析:

1. 大语言模型具备强大的语义理解能力, 能够从上下文中正确关联描述性词语和其所修饰的对象。对于明确表达的属性和情感, 如评论 1 的“服务员态度”为“正面”, “上菜速度”为“负面”; 评论 2 的“环境”、“灯光”为“正面”, “位置”为“负面”; 评论 4 的“食物”为“正面”, 模型均能准确捕捉。
2. 模型对自然语言的理解深入, 能够识别出更具体的评价对象。例如模型能够提取出如“服务员态度”(评论 1)、“排队时长”(评论 4)、“菜的分量”(评论 5)、“环境(吵闹情况)”(评论 6)等较为细致的属性, 而不仅仅是泛指的“服务”或“环境”。
3. 模型词汇库丰富, 对情感强度和倾向的理解较为细致。对于评论 3 “味道一般”, 模型给出了“味道”: “中性”的判断, 这是非常准确的, 因为“一般”通常表示中性的评价。
4. 模型的训练数据包含了大量真实世界的语料, 使其能够进行一定程度的常识性推断和语境关联。评论 5 “虽然等位时间长了一点, 但考虑到是周末也可以理解”, 模型仍将“等位时间”判断为“负面”, 这抓住了核心的负面体验, 尽管评论者表示了理解。评论 6 中, 将“人流量”判断为“负面”, 并将“吵闹”归纳为“环境(吵闹情况)”的负面, 是模型将“人流量巨大”与负面体验“吵闹”进行了关联。

进阶探索

```
1 '''
2 # 角色
3 你是一个专业的中文属性级情感分析助手。你的任务是仔细阅读用户提供的评论文本，进行细致的分析，最终
  准确识别出评论中提及的所有评价属性，并判断针对每个属性的情感极性（正面、负面或中性）。
4
5 # 任务描述
6 请你按照“思维链”（Chain-of-Thought）的方式来分析用户评论。对于每一条评论，你的分析应包含以下步
  骤：
7 1.  **句子分析与属性识别**：首先，请分解评论句子，识别出其中明确提及的、用户发表了看法的具体方面
  或特征（即“属性”）。列出相关的描述性短语和它们对应的属性。
8 2.  **情感判断与理由**：接着，对每个识别出的属性，根据相关的描述性词语或短语，判断其所表达的情感
  极性（正面、负面或中性），并简要说明判断理由。
9 3.  **最终JSON输出**：最后，将**这条评论的**分析结果汇总为严格的JSON格式。
10
11 # 输出格式要求
12 对于“需要你逐条分析的评论”列表中的每一条评论，都请严格按照#示例中展示的完整格式进行输出。即，每
  一条评论的输出都应包含其独立的“分析过程”（包含步骤1“句子分析与属性识别”和步骤2“情感判断
  与理由”）和紧随其后的步骤3“最终JSON输出”。
13 **重要：不要将所有评论的JSON结果合并在一个总的JSON对象中。每一个“最终JSON输出”都只针对其对应的
  那一条评论。**
14
15 # 示例
16
17 ## 示例1
18 用户评论：这家餐厅的食物很好吃，但是价格有点贵。
19
20 分析过程：
21 1. 句子分析与属性识别：
22     - 从“食物很好吃”中，识别出属性：“食物”。
23     - 从“价格有点贵”中，识别出属性：“价格”。
24 2. 情感判断与理由：
25     - 对于“食物”，描述为“很好吃”，表达了积极的用餐体验，因此情感为“正面”。
26     - 对于“价格”，描述为“有点贵”，表达了对花费的负面感受，因此情感为“负面”。
27 3. 最终JSON输出：
28     {"食物": "正面", "价格": "负面"}
29
30 ## 示例2
31 用户评论：服务员态度非常好，就是上菜速度太慢了。
32
33 分析过程：
34 1. 句子分析与属性识别：
35     - 从“服务员态度非常好”中，识别出属性：“服务员态度”。
36     - 从“上菜速度太慢了”中，识别出属性：“上菜速度”。
37 2. 情感判断与理由：
38     - 对于“服务员态度”，描述为“非常好”，表达了对服务的强烈满意，因此情感为“正面”。
39     - 对于“上菜速度”，描述为“太慢了”，表达了对效率的不满，因此情感为“负面”。
40 3. 最终JSON输出：
41     {"服务员态度": "正面", "上菜速度": "负面"}
42
43 ## 示例3
44 用户评论：环境不错，灯光也很有氛围，就是位置偏了点。
```

```
45 分析过程：
46
47 1. 句子分析与属性识别：
48   - 从“环境不错”中，识别出属性：“环境”。
49   - 从“灯光也很有氛围”中，识别出属性：“灯光”。
50   - 从“就是位置偏了点”中，识别出属性：“位置”。
51 2. 情感判断与理由：
52   - 对于“环境”，描述为“不错”，表达了积极的感受，因此情感为“正面”。
53   - 对于“灯光”，描述为“很有氛围”，表达了积极的感受，因此情感为“正面”。
54   - 对于“位置”，描述为“偏了点”，表达了对便利性的负面感受，因此情感为“负面”。
55 3. 最终JSON输出：
56   {"环境": "正面", "灯光": "正面", "位置": "负面"}
57
58 # 需要你逐条分析的评论
59 以下是需要你分析的评论列表。请为列表中的每一条评论，都完整地复现上述#示例中的输出结构（即每条评论
60   都有自己的“分析过程”和紧随其后的“最终JSON输出”，如示例所示）。
61
62 用户评论列表：
63 评论1：这家餐厅的食物很好吃，但是价格有点贵。
64 评论2：服务员态度非常好，就是上菜速度太慢了。
65 评论3：环境不错，灯光也很有氛围，就是位置偏了点。
66 评论4：味道一般，价格还算合理。
67 评论5：排队太久了，但食物真的物有所值。
68 评论6：这家店的装修很有特色，饮品味道尚可，但甜点不推荐，太甜了。
69 评论7：地理位置非常方便，就在地铁口旁边，不过高峰期人流量巨大，有点吵闹。
70 评论8：菜品种类挺丰富的，海鲜很新鲜，价格也公道，就是包间有点小。
71 评论9：整体体验一般，客服回复及时，物流速度也还可以，但产品包装有些简陋。
72 评论10：虽然等位时间长了一点，但考虑到是周末也可以理解，菜的分量很足，性价比不错。
73
74 # 模型输出开始：
75 （模型应从这里开始，为评论1生成包含“分析过程”和“最终JSON输出”的完整块，然后为评论2生成同样的完
   整块，以此类推，直到处理完列表中的所有评论。）
'''
```

Listing 4: 优化后的提示词

采用豆包进行评估：

1. 这家餐厅的食物很好吃，但是价格有点贵。

分析过程:

1. 句子分析与属性识别:

- 从“这家餐厅的食物很好吃”中，识别出属性：“食物”。
- 从“但是价格有点贵”中，识别出属性：“价格”。

2. 情感判断与理由:

- 对于“食物”，描述为“很好吃”，表达了积极的用餐体验，因此情感为“正面”。
- 对于“价格”，描述为“有点贵”，表达了对花费的负面感受，因此情感为“负面”。

3. 最终 JSON 输出:

{"食物": "正面", "价格": "负面"}

2. 服务员态度非常好，就是上菜速度太慢了。

分析过程:

1. 句子分析与属性识别:

- 从“服务员态度非常好”中，识别出属性：“服务员态度”。
- 从“就是上菜速度太慢了”中，识别出属性：“上菜速度”。

2. 情感判断与理由:

- 对于“服务员态度”，描述为“非常好”，表达了对服务的强烈满意，因此情感为“正面”。
- 对于“上菜速度”，描述为“太慢了”，表达了对效率的不满，因此情感为“负面”。

3. 最终 JSON 输出:

{"服务员态度": "正面", "上菜速度": "负面"}

3. 环境不错，灯光也很有氛围，就是位置偏了点。

分析过程:

1. 句子分析与属性识别:

- 从“环境不错”中, 识别出属性: “环境”。
- 从“灯光也很有氛围”中, 识别出属性: “灯光”。
- 从“就是位置偏了点”中, 识别出属性: “位置”。

2. 情感判断与理由:

- 对于“环境”, 描述为“不错”, 表达了积极的感受, 因此情感为“正面”。
- 对于“灯光”, 描述为“很有氛围”, 表达了积极的感受, 因此情感为“正面”。
- 对于“位置”, 描述为“偏了点”, 表达了对便利性的负面感受, 因此情感为“负面”。

3. 最终 JSON 输出:

```
{"环境": "正面", "灯光": "正面", "位置": "负面"}
```

4. 味道一般, 价格还算合理。

分析过程:

1. 句子分析与属性识别:

- 从“味道一般”中, 识别出属性: “味道”。
- 从“价格还算合理”中, 识别出属性: “价格”。

2. 情感判断与理由:

- 对于“味道”, 描述为“一般”, 表达的情感为中性, 没有明显的积极或消极倾向。
- 对于“价格”, 描述为“还算合理”, 表达了对价格的认可, 因此情感为“正面”。

3. 最终 JSON 输出:

```
{"味道": "中性", "价格": "正面"}
```

5. 排队太久了, 但食物真的物有所值。

分析过程：

1. 句子分析与属性识别：

- 从“排队太久了”中，识别出属性：“排队情况”。
- 从“但食物真的物有所值”中，识别出属性：“食物”。

2. 情感判断与理由：

- 对于“排队情况”，描述为“太久了”，表达了对排队时间的不满，因此情感为“负面”。
- 对于“食物”，描述为“真的物有所值”，表达了对食物价值的高度认可，因此情感为“正面”。

3. 最终 JSON 输出：

{“排队情况”: “负面”, “食物”: “正面”}

6. 这家店的装修很有特色，饮品味道尚可，但甜点不推荐，太甜了。

分析过程：

1. 句子分析与属性识别：

- 从“这家店的装修很有特色”中，识别出属性：“装修”。
- 从“饮品味道尚可”中，识别出属性：“饮品味道”。
- 从“但甜点不推荐，太甜了”中，识别出属性：“甜点”。

2. 情感判断与理由：

- 对于“装修”，描述为“很有特色”，表达了对装修的赞赏，因此情感为“正面”。
- 对于“饮品味道”，描述为“尚可”，表达的情感为中性，没有明显的积极或消极倾向。
- 对于“甜点”，描述为“不推荐，太甜了”，表达了对甜点的不同意，因此情感为“负面”。

3. 最终 JSON 输出：

{“装修”: “正面”, “饮品味道”: “中性”, “甜点”: “负面”}

7. 地理位置非常方便，就在地铁口旁边，不过高峰期人流量巨大，有点吵闹。

分析过程：

1. 句子分析与属性识别：

- 从“地理位置非常方便，就在地铁口旁边”中，识别出属性：“地理位置”。
- 从“不过高峰期人流量巨大，有点吵闹”中，识别出属性：“人流量”和“环境吵闹程度”。

2. 情感判断与理由：

- 对于“地理位置”，描述为“非常方便”，表达了对位置便利性的高度认可，因此情感为“正面”。
- 对于“人流量”，描述为“巨大”，表达了对人流量过多的负面感受，因此情感为“负面”。
- 对于“环境吵闹程度”，描述为“有点吵闹”，表达了对环境噪音的负面感受，因此情感为“负面”。

3. 最终 JSON 输出：

{“地理位置”: “正面”, “人流量”: “负面”, “环境吵闹程度”: “负面”}

8. 菜品种类挺丰富的，海鲜很新鲜，价格也公道，就是包间有点小。

分析过程：

1. 句子分析与属性识别：

- 从“菜品种类挺丰富的”中，识别出属性：“菜品种类”。
- 从“海鲜很新鲜”中，识别出属性：“海鲜”。
- 从“价格也公道”中，识别出属性：“价格”。
- 从“就是包间有点小”中，识别出属性：“包间大小”。

2. 情感判断与理由：

- 对于“菜品种类”，描述为“挺丰富的”，表达了对菜品种类的满意，因此情感为“正面”。
- 对于“海鲜”，描述为“很新鲜”，表达了对海鲜品质的认可，因此情感为“正面”。
- 对于“价格”，描述为“公道”，表达了对价格的认可，因此情感为“正面”。
- 对于“包间大小”，描述为“有点小”，表达了对包间空间的不满意，因此情感为“负面”。

3. 最终 JSON 输出：

```
{"菜品种类": "正面", "海鲜": "正面", "价格": "正面", "包间大小": "负面"}
```

9. 整体体验一般，客服回复及时，物流速度也还可以，但产品包装有些简陋。

分析过程：

1. 句子分析与属性识别：

- 从“整体体验一般”中，识别出属性：“整体体验”。
- 从“客服回复及时”中，识别出属性：“客服回复速度”。
- 从“物流速度也还可以”中，识别出属性：“物流速度”。
- 从“但产品包装有些简陋”中，识别出属性：“产品包装”。

2. 情感判断与理由：

- 对于“整体体验”，描述为“一般”，表达的情感为中性，没有明显的积极或消极倾向。
- 对于“客服回复速度”，描述为“及时”，表达了对客服服务的认可，因此情感为“正面”。
- 对于“物流速度”，描述为“还可以”，表达的情感为中性，没有明显的积极或消极倾向。
- 对于“产品包装”，描述为“有些简陋”，表达了对产品包装的不满意，因此情感为“负面”。

3. 最终 JSON 输出：

```
{"整体体验": "中性", "客服回复速度": "正面", "物流速度": "中性", "产品包装": "负面"}
```

10. 虽然等位时间长了一点，但考虑到是周末也可以理解，菜的分量很足，性价比不错。

分析过程：

1. 句子分析与属性识别：

- 从“虽然等位时间长了一点”中，识别出属性：“等位时间”。
- 从“但考虑到是周末也可以理解”中，此句主要是对“等位时间长”的解释，未产生新的评价属性。
- 从“菜的分量很足”中，识别出属性：“菜的分量”。
- 从“性价比不错”中，识别出属性：“性价比”。

2. 情感判断与理由：

- 对于“等位时间”，描述为“长了一点”，表达了对等位时间的不满，因此情感为“负面”。
- 对于“菜的分量”，描述为“很足”，表达了对菜量的满意，因此情感为“正面”。
- 对于“性价比”，描述为“不错”，表达了对性价比的认可，因此情感为“正面”。

3. 最终 JSON 输出：

{“等位时间”: “负面”, “菜的分量”: “正面”, “性价比”: “正面”}

改进方案相较于基础方案，主要优势在于：通过思维链分析过程，提供了极高的可解释性，并在处理中性或模糊情感时表现更稳定、准确。

- **思维链提示**在这个任务中非常有效。它不仅提升了模型处理复杂、模糊语句的准确性，更关键的是它提供了宝贵的可解释性，让用户能理解模型判断的依据。对于需要深入分析和洞察的 ABSA 任务来说，这种透明度至关重要。同时，它还有助于约束模型按步骤思考，间接提升了遵循复杂格式指令的能力。然而，思维链有可能增加 Prompt 长度和 API 调用成本，因此需要在性能收益和成本之间权衡。
- **增强的 Few-shot 学习**提供了多个覆盖不同情况的示例，是引导 LLM 学习特定任务模式和输出格式的基础且高效的方法。因此有较高的推广价值。
- 无论任务简单或复杂，**清晰、无歧义地描述任务目标和输出格式**都是成功 Prompt 的基础。